

Numberlink

PRM2T.25L

Bartosz Skrzypczak

Natalia Regulska

Jakub Zagrabski

Kajetan Rosik

2025

Contents

1	Wstęp	3
2	Etap 1	3
2.1	Podział zadań	3
2.2	Opis słowny przypadków użycia	4
2.3	Diagram przypadków użycia	4
2.4	Diagram klas	5
2.5	Repozytorium	5
3	Etap 2	5
3.1	Założenia etapu	5
3.2	Opis działania kodu	5
3.3	Wnioski na dalsze etapy gry	6
4	Etap 3	6
4.1	Założenia etapu	6
4.2	Dodanie opcji cofania ruchów	7
4.3	Poprawa błędu	7
4.4	Docelowe działanie gry	7
4.4.1	Klasa Level	7
4.4.2	Klasa LevelManager	7
4.4.3	Klasa NumberLinkV4	7
4.4.4	Klasa Colors	8
4.4.5	Klasa Menu	8
4.4.6	Klasa GameFrame	8
4.5	Ostatnie poprawki	8
4.6	Dodanie opcji wstecz	8
4.7	Dodanie opcji pobrania planszy w formacie .png	8
5	Problemy, które napotkaliśmy po drodze	8

1 Wstęp

Celem projektu jest stworzenie i zaprogramowanie gry-łamigłówki "Number-link".

Zasady są bardzo proste. Należy połączyć pary kafelków z numerkami pojedynczymi nieprzerwanymi liniami. Linie nie mogą się poruszać po skosie, przecinać innych linii, najeżdżać na inne numerki i nie mogą zostawiać pustych miejsc.

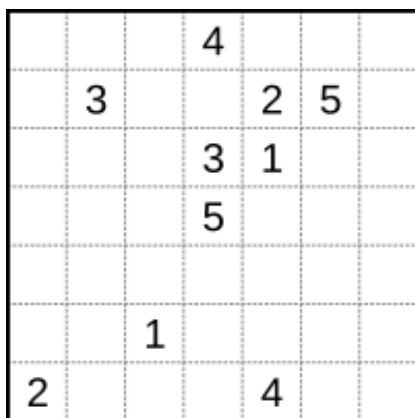


Fig. 1: Przykładowa pusta plansza

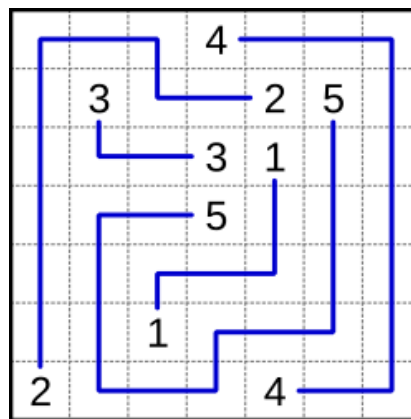


Fig. 2: Przykładowa plansza po rozwiązaniu

2 Etap 1

2.1 Podział zadań

Spotykamy się raz w tygodniu, aby omówić kolejne kroki podejmowane w celu implementacji gry NumberLink. W ciągu tygodnia zbieramy pomysły, a następnie dzielimy się naszymi przemyśleniami. Dzielimy się zadaniami:

- Sprawozdanie w Latex oraz ogólna funkcjonalność projektu -Kajetan Rosik
- Implementacja sprawdzania warunków gry oraz poprawności połączenia numerków (klasa Sprawdzanie) - Jakub Zagrabski
- Koordynator zespołu oraz sprawdzanie poprawności połączenia numerków (klasa Sprawdzanie) - Bartosz Skrzypczak
- Tworzenie plansz, poziomów oraz implementacja klasy Komorka - Natalia Regulska

2.2 Opis słowny przypadków użycia

Potrzebne jest aby gracz miał wpływ na to co się dzieje, żeby komunikacja z aplikacją przebiegała bez problemu.

- Użytkownik naciska przycisk "Start", poziomy zostają wczytane,
- Użytkownik wybiera planszę, plansza zostanie wczytana,
- Podjęta jest próba rozwiązania planszy, a program sprawdza rozwiązanie pod względem poprawności,
- W momencie zwycięstwa, jest to zakomunikowane graczowi a bieg programu wraca do "menu" czyli etapu wyboru poziomu. Koło się zamyka. W momencie porażki, jest ona zakomunikowana graczowi i nic więcej się nie dzieje,
- Dodatkowo obecna jest opcja zamknięcia programu w każdej chwili, przyciskiem "zakończ".

2.3 Diagram przypadków użycia

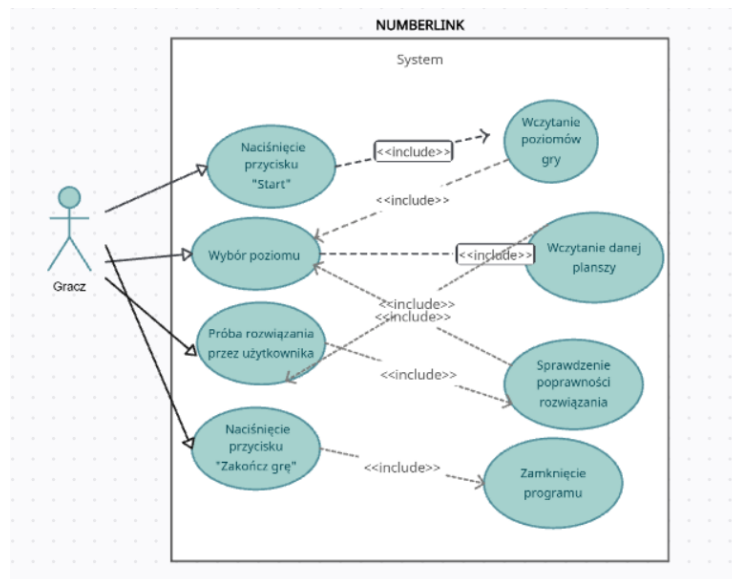


Fig. 3: Diagram użycia klas

2.4 Diagram klas

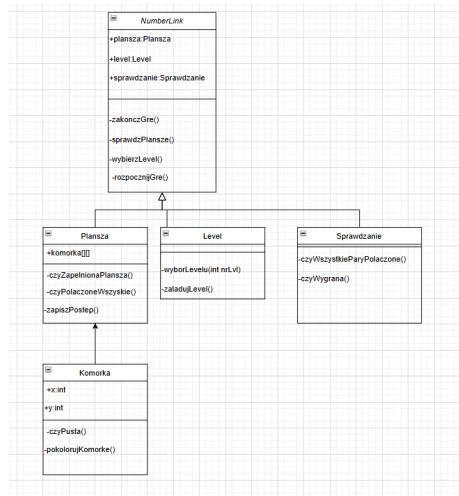


Fig. 4: Diagramy klas

2.5 Repozytorium

Utworzyliśmy repozytorium w gitlabie, do których dodaliśmy prowadzących.

3 Etap 2

3.1 Założenia etapu

W tym etapie chcieliśmy zaimplementować ogólny zarys gry, aby sprawdzić co może nam sprawić trudności w ostatecznej implementacji. Dlatego postanowiliśmy stworzyć prostą planszę testową, którą będzie można rozwiązać. Na jej podstawie będziemy rozwijać dalsze działanie naszej gry w ostatnim etapie.

3.2 Opis działania kodu

- **Inicjalizacja i rysowanie planszy** - konstruktor klasy przyjmuje dwuwymiarową tablicę `board`, która definiuje pozycje numerków do połączenia oraz interfejs `Runnable` wykonywany po ukończeniu poziomu. Plansza jest tworzona jako siatka kwadratowych komórek o ustalonym rozmiarze (`cellSize = 80`). Kolory są przypisane do numerków za pomocą mapy `colorMap`. Metoda `paintComponent` odpowiada za graficzne renderowanie planszy. Wypełnia komórki odpowiednim kolorem (zależnie od numerka lub koloru linii) i rysuje siatkę oraz numery na planszy.

- **Przesuwanie myszką** - zaimplementowany został adapter myszy MouseAdapter, który obsługuje trzy zdarzenia:
 - **MousePressed** - wykrywa kliknięcie na numerku startowym i inicjalizuje nową ścieżkę (currentPath) od tego punktu.
 - **MouseDragged** - podczas przeciągania myszy, jeśli nowy punkt jest sąsiadem poprzedniego i spełnia warunki (pusta komórka lub ten sam numer), komórka zostaje pokolorowana, a ścieżka wydłużona.
 - **MouseReleased** - po puszczeniu przycisku myszy wywoływana jest funkcja checkWin, która sprawdza, czy wszystkie puste komórki zostały wypełnione – jeśli tak, użytkownik otrzymuje komunikat o wygranej.
- **Logika gry:**
 - **isAdjacent** - metoda pomocnicza sprawdzająca, czy dwa punkty sąsiadują w pionie lub poziomie.
 - **checkWin** - przechodzi przez wszystkie komórki planszy i zwraca true, tylko jeśli nie pozostały żadne puste komórki, które powinny być połączone.
- **main - uruchomienie gry** - metoda main inicjalizuje prostą planszę 4x4 i tworzy okno gry przy użyciu JFrame. Przycisk "zamknięcia" zamyka aplikację, a plansza jest renderowana na środku ekranu. Po ukończeniu poziomu (poprawnym połączeniu wszystkich numerków), w konsoli pojawia się komunikat „Level complete”.

3.3 Wnioski na dalsze etapy gry

Przy tym etapie napotkaliśmy pewne problemy z implementacją kodu dotyczącego przesuwania myszką oraz zarządzaniem kolorem i rysowaniem ścieżek. W tym etapie uprościliśmy nasz kod, ponieważ nie da się jeszcze cofać ruchu. Ta funkcja zostanie dodana w ostatnim etapie. W tym etapie jednak udało nam się zrealizować nasze główne założenia i mamy fundament do pracy nad dalszym rozwojem gry.

4 Etap 3

4.1 Założenia etapu

W tym etapie chcieliśmy zaimplementować cofanie ruchów, aby uczynić grę bardziej przyjazną dla użytkownika, i wczytywanie plansz z pliku, ze względu na prostotę dodawania następnych poziomów.

4.2 Dodanie opcji cofania ruchów

W pierwszej kolejności dodaliśmy opcję cofania ruchów. Wystarczy kliknąć na bazowe pole z cyferką, a ścieżka zostaje usunięta. W tym celu zmodyfikowaliśmy metodę **mousePressed**. Sprawdzamy, czy kliknięty numer to ten sam, którego ścieżka już istnieje. Jeżeli tak, to przeszukujemy tablicę i czyścimy pola z tym numerem. Następnie zerujemy **currentPath**, **selectedNumber**, **lastPoint**.

4.3 Poprawa błędu

Podczas dalszej implementacji zauważyliśmy bardzo duży błąd, który popełniliśmy. Nasza metoda **checkWin** była niekompletna i nieprawidłowo sprawdzała planszę. Mogliśmy zamalować całą planszę jednym kolorem, a gra i tak była wygrana. Teraz sprawdzamy, czy para liczb jest połączona dokładnie jedną ścieżką. Musieliśmy dodać także pomocniczą metodę **isConnected**, która przechodzi tylko po polach wypełnionych odpowiednim kolorem. Korzystaliśmy tutaj z wiedzy z laboratorium 4, gdzie musieliśmy w podobny sposób wyznaczać liczbę białych obszarów w obrazie.

4.4 Docelowe działanie gry

Musieliśmy bardziej rozbudować nasz kod, aby zaimplementować wczytywanie poziomów. Poziomy wczytujemy z pliku **poziomy.txt**. Plik składa się z 9 poziomów. Aby ułatwić sobie wczytywanie ich, pierwsza linijka to zawsze tytuł poziomu zapisany w cudzysłowie, a następnie zapisujemy liczby oddzielone spacją, a następnie przecinkiem. 0 oznacza puste pole, natomiast każda inna cyfra oznacza bazowe pole do planszy.

4.4.1 Klasa Level

Jest to bazowa klasa do przechowywania poziomu. Przechowujemy tutaj nazwę poziomu oraz pola planszy w postaci tablicy.

4.4.2 Klasa LevelManager

Ta klasa wczytuje poziomy z pliku i przechowuje je w formie tablicy z poprzedniej klasy Level. Tutaj metoda **loadLevels** wczytuje plik i wg zasad które opisałam na początku sekcji, odczytuje nazwę nowego poziomu i parsuje linie do tablicy.

4.4.3 Klasa NumberLinkV4

Z tej klasy usunęliśmy maina. Będzie on teraz w osobnej klasie, która będzie odpowiadała za uruchamianie gry.

4.4.4 Klasa Colors

Służy tylko to przechowywania nowych kolorów planszy, ponieważ dla 10 numerków było lub zbyt mało lub nie podobały nam się domyślne kolory.

4.4.5 Klasa Menu

Ta klasa odpowiada za GUI. Wyświetlane są przyciski z nazwami poziomów oraz ogólny wygląd gry. Dodajemy także scrolla, ponieważ nie mieściły nam się przyciski.

4.4.6 Klasa GameFrame

To jest nasz main. Mamy tutaj konstruktor, który ustawia podstawowe elementy okna. Mamy metodę **showMenu**, która tworzy nowy obiekt Menu, ustawia go na główny widok, a na koniec odświeża GUI. Mamy też metodę **startGame**, która ustawia nazwę okna na nazwę poziomu, tworzy obiekt klasy NumberLinkV4 i ustawia widok okna.

4.5 Ostatnie poprawki

4.6 Dodanie opcji wstecz

Następnie do klasy **NumberLinkV4** dodaliśmy opcję, która przy naciśnięciu klawisza **Esc** będzie cofała nas do menu wyboru poziomu.

4.7 Dodanie opcji pobrania planszy w formacie .png

W klasie **GameFrame** dodaliśmy przycisk, który pojawia się przy każdej planszy po wybraniu poziomu. Będzie on zapisywał zrzut ekranu planszy do pliku.

5 Problemy, które napotkaliśmy po drodze

Pierwszym problemem, który się pojawił była opcja poprawnego sprawdzania planszy. Przy testach okazywało się, że pomijaliśmy różne ważne aspekty i zajęło nam sporo czasu poprawne zaimplementowanie tej funkcji. Kolejną rzeczą, która zajęła nam dużo czasu było rozplanowanie, jak wczytywać poziomy oraz jak napisać kod, aby można było łatwo wprowadzać poprawki. Ostatnią rzeczą, która przysporzyła nam problemy był zapis poprzedniego stanu gry. Niestety te zadanie było dla nas zbyt trudne i nie udało nam się go wykonać.